Task 1: Load Model & Count Trainable Parameters with LoRA
Here is the complete Python code using Hugging Face transformers and peft to print trainable parameters before and after applying LoRA.
python

```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from peft import LoraConfig, get_peft_model

# 1. Define model name (Sentiment Analysis model)
model_name = "distilbert-base-uncased-finetuned-sst-2-english"

# 2. Load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)

def print_trainable_parameters(model, label=""):
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    all_params = sum(p.numel() for p in model.parameters())
    print(f"[{label}] Trainable params: {trainable_params:,} | All params: {all_params:,} | Ratio:
{100 * trainable_params / all_params:.4f}%")

# 3. Print parameters BEFORE LoRA
print_trainable_parameters(model, "BEFORE LoRA")

# 4. Configure LoRA
peft_config = LoraConfig(
    task_type="SEQ_CLS",
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=["q_lin", "v_lin"] # Specific to DistilBERT attention layers
)

# 5. Apply LoRA using get_peft_model
peft_model = get_peft_model(model, peft_config)

# 6. Print parameters AFTER LoRA
print_trainable_parameters(peft_model, "AFTER
```

Task 2: Fine-Tuning on a Custom Dataset & Checking File Size
This script defines a 10-tweet toy dataset, sets up a minimal training pipeline, saves the adapter weights, and prints the resulting file size.

```
python
import os
import torch
from datasets import Dataset
from transformers import AutoModelForSequenceClassification, AutoTokenizer,
TrainingArguments, Trainer
from peft import LoraConfig, get_peft_model, TaskType

# 1. Setup Model & Tokenizer
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)

# 2. Define 10 sample tweets (0 = Negative, 1 = Positive)
data = {
    "text": [
        "I love this product, it works perfectly!", "Worst experience ever, totally broken.",
        "Incredible service and friendly staff.", "Waste of money, do not buy this.",
        "Absolutely amazing, highly recommend!", "It is okay, nothing special but works.",
        "Terrible quality, deeply disappointed.", "Super fast delivery and great quality.",
        "The app keeps crashing, useless.", "This is the best purchase I have made all year!"
    ],
    "label": [1, 0, 1, 0, 1, 1, 0, 1, 0, 1]
}
dataset = Dataset.from_dict(data)

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True,
max_length=32)

tokenized_dataset = dataset.map(tokenize_function, batched=True)

# 3. Apply LoRA Configuration
peft_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=["q_lin", "v_lin"]
)
peft_model = get_peft_model(model, peft_config)

# 4. Define Minimal Training Arguments
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=2,
```

```
    num_train_epochs=3,
    weight_decay=0.01,
    logging_steps=1,
)

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=tokenized_dataset,
)

# 5. Fine-tune the model
trainer.train()

# 6. Save adapted weights to disk
output_dir = "./lora_saved_weights"
peft_model.save_pretrained(output_dir)
print(f"Weights successfully saved to '{output_dir}'")

# 7. Check and report file size
adapter_model_path = os.path.join(output_dir, "adapter_model.safetensors")
if os.path.exists(adapter_model_path):
    file_size_mb = os.path.getsize(adapter_model_path) / (1024 * 1024)
    print(f"Size of adapter_model.safetensors: {
```

Task 3: Concept & Real-World Use Cases

Main Benefits of PEFT over Full Fine-Tuning

• Drastically Reduced Storage: Instead of saving a massive multi-gigabyte copy of the base model for every task, PEFT only saves low-rank adapter weights. This reduces storage needs up to 99% (often only a few megabytes per task).

• Lower Computational Costs: Freezing the base model means we do not compute or update gradients for billions of parameters, massively lowering VRAM/RAM overhead and accelerating training.

• Mitigates Catastrophic Forgetting: Because the core base model parameters remain entirely untouched, the model retains its original generalized knowledge while learning the new niche behavior via adapters.

Real-World App Examples

1. Instagram (Multi-Task Content Moderation & Tagging)

    • Why PEFT is useful: Instagram handles distinct, concurrent NLP tasks: toxic comment filtering, automatic image caption translation, and user interest tagging.

    • How it helps: Instead of serving three independent 7-billion parameter models (which would cost huge amounts of server infrastructure), Instagram can host one base model in memory and dynamically swap ultra-lightweight PEFT adapters depending on whether a user is commenting, translating, or exploring content.

2. Zomato (Hyper-Personalized Review Summarization)

• Why PEFT is useful: Zomato could customize response and summary styles dynamically based on localized regions, diet preferences (vegan vs. non-veg critics), or individual restaurant cuisines (Fine Dining vs. Street Food).

• How it helps: Creating a fully fine-tuned LLM for every single culinary category or city is impossible. PEFT allows Zomato to quickly train and deploy small local adapters that modify a single core model's tone to match specific consumer categories.

Task 4: QLoRA Fine-Tuning & Memory Monitoring

To implement QLoRA, you need to load the model in 4-bit quantization using bitsandbytes.

Code Setup for QLoRA

python

```
from transformers import BitsAndBytesConfig, AutoModelForSequenceClassification

# Define 4-bit quantization configuration
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)

# Load base model with the quantization config
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    quantization_config=bnb_config,
    device_map="auto"
)
```

Use code with caution.

How to complete the monitoring step:

1. Open Task Manager (Windows) or run top / htop (Linux/Mac) side by side with your code window.
2. Observe your system RAM/VRAM usage before execution.
3. Run standard full fine-tuning (Task 2 without freezing parameters) and take a screenshot of your peak memory usage graph.
4. Run the QLoRA script and take a second screenshot to visually display the lower, flattened memory utilization footprint.

Task 5: LLM Code Generation & Reflection

To complete this task, pass the prompt below to ChatGPT or Copilot:

"Write a Python script using Hugging Face transformers and peft to apply LoRA to a text classification task using BertForSequenceClassification."

Example of 2-3 Lines Reflection (Write-up sample):

"The code generated by the LLM initially targeted ['q', 'v'] modules which caused an execution error because BERT uses query and value names inside its self-attention layers. I modified the target_modules list to match BERT's exact architecture and added a padding token configuration to prevent an index out-of-range error during batch tokenization."